UCS420: Cognitive Computing

Assignment 4

A Cognitive FAQ System Using Pandas (Nova 2.0)

Name: Krish Marwaha
Roll No: 1024170223
Batch: 3Q24

Q1. Build Your Personalized Knowledge Base

Take the last two digits of roll number 1024170223: 2 and 3. For digit 2, 2 % 3 = 2, so category is general. For digit 3, 3 % 3 = 0, so category is billing.

```
import pandas as pd

fixed_entries = [
    {"question": "what is the annual fee",
     "answer": "The annual fee is Rs 500.",
     "keywords": "fee cost price charge", "category": "billing"},

    {"question": "how to reset password",
     "answer": "Go to Settings > Reset Password.",
     "keywords": "password reset login", "category": "account"},

    {"question": "what are your working hours",
     "answer": "We are open 9 AM to 5 PM.",
     "keywords": "hours timing open time", "category": "general"},

    {"question": "how can i pay the fee",
     "answer": "You can pay via UPI, card, or net banking.",
     "keywords": "pay payment upi fee", "category": "billing"}
]

personalized_entries = [
    {"question": "how can i contact support",
     "answer": "You can contact support by email or phone.",
     "keywords": "support help contact", "category": "general"},

    {"question": "how can i check my payment status",
     "answer": "You can check payment status in your account.",
     "keywords": "payment status transaction", "category": "billing"}
]

faq = fixed_entries + personalized_entries
df = pd.DataFrame(faq)

print(df)
```

Q2. Generate and Score a Hypothesis

Count how many query words are found in the question or keywords. Print matching entries from highest score to lowest score.

```
def score_query(query, df):
    query_words = query.lower().split()
    results = []

    for i, row in df.iterrows():
        text = (row["question"] + " " + row["keywords"]).lower()
        score = 0

        for word in query_words:
            if word in text:
                score += 1

        if score > 0:
            results.append((score, i))

    results.sort(reverse=True)

    for score, i in results:
        print("Score:", score)
        print("Question:", df.loc[i, "question"])
        print("Answer:", df.loc[i, "answer"])
        print()

score_query("reset password", df)
```

Q3. Find Entries from the Same Category

```
def same_category(category_name, df):
    return df[df["category"] == category_name][["question", "category"]]

print(same_category("general", df))
```

Q4. Add a New Keyword and Save the CSV

```
index = 0

new_keyword = input("Enter a new keyword: ")

df.loc[index, "keywords"] += " " + new_keyword

df.to_csv("1024170223_faq_data.csv", index=False)

print(df)
print("File saved as 1024170223_faq_data.csv")
```

Q5. Count FAQ Entries Per Category

```
category_count = df.groupby("category").size()

print(category_count)
```

Output: billing = 3, general = 2, account = 1.

Q6. Handle Ties in the Highest Score

If several entries have the same highest score, print all of them.

```
def score_query_tie(query, df):
    query_words = query.lower().split()
    scores = []

    for i, row in df.iterrows():
        text = (row["question"] + " " + row["keywords"]).lower()
        score = 0

        for word in query_words:
            if word in text:
                score += 1

        scores.append((score, i))

    max_score = max(score for score, i in scores)

    if max_score == 0:
        print("No matching entry")
        return

    print("Highest score:", max_score)

    for score, i in scores:
        if score == max_score:
            print("Question:", df.loc[i, "question"])
            print("Answer:", df.loc[i, "answer"])
            print()

score_query_tie("fee", df)
score_query_tie("password", df)
```